

Speaker Script

An Automated Framework for Dataset Quality Assessment and Data Leakage Detection in Machine Learning

25-minute defense — matches `presentation.pptx` (30 slides)
 Texto hablado en **inglés** · acotaciones en *cursiva gris* en español

Antes de empezar — chuleta de números (si dudas, di el número redondo):

Contexto inicial (nuevo): adopción IA **88%** en 2025 (vs. **50%** en 2022) · mercado IA **\$260B** → **\$1.2T+** (2025→2030) · coste medio mala calidad de datos **\$12.9M/año/organización**.

Núcleo de la tesis: **20** checks / **5** dimensiones (el **29** es el *checklist* del benchmark, no lo mezcles) · **12** datasets (6+6) · **343** tests · **4/4** escenarios de leakage vs. 0–1/4 · accuracy **1.000** → **0.952** ($\Delta = -0.048$) · boat $\mathcal{L} = \mathbf{0.74}$ (ablation: cae a 0.662 con esquema correlation-heavy) · name Cramér's V = **0.999** · Titanic **80.6/B** · umbral warning **0.7** / error **0.9** · pesos LRS **0.35/0.35/0.30** · pesos readiness **0.25/0.35/0.25/0.15**.

Módulo semántico (evaluación en vivo, Claude Haiku 4.5 / AWS Bedrock): umbral medium P/R/F1 = **1.00/1.00/1.00** · umbral high P/R/F1 = **0.917/0.846/0.880** · 3 discrepancias reales en high (treatment_count, batch_rejection_rate, training_completion_post_hire).

Ritmo: ≈ 110 –115 palabras/min. Punto de control: en la slide 17 ("From 20 Checks to One Number") deberías ir por ≈ 14 –15 min. Si vas por encima, comprime las slides 11 (Interfaces) y 24 (Feature Coverage) a una frase, o recorta la narración de la demo a 2 min en vez de 3.

Part 1 — Motivation & Objectives ($\approx 7:00$)

Slide 1 — Title

0:20

acum. 0:20

De pie, contacto visual, sonríe. No leas el título entero.

Good morning. My name is Jaime Cano Morano, and today I will present my master's thesis: an automated framework for dataset quality assessment and data leakage detection in machine learning.

Slide 2 — Agenda

0:40

acum. 1:00

Señala las cinco secciones con la mano, no las leas palabra por palabra.

In this talk, I will cover five parts. First, why machine learning matters today and why data leakage is such a critical problem. Second, the system I built and the methodology behind it — this is the core of the talk. Third, the experimental results on twelve datasets. Fourth, a quantitative benchmark against three established tools. And finally, a short live demo, conclusions, and future work.

Slide 3 — Divider 01

0:10

acum. 1:10

Pasa rápido; una sola frase mientras cambia.

Let us start with the motivation.

Slide 4 — Why Machine Learning Matters Today**1:10****acum. 2:20**

Slide nueva de contexto. Señala cada KPI mientras lo dices; no leas la gráfica entera, solo el 2022 y el 2025.

Before diving into leakage, let me set the stage. Machine learning is no longer a niche research topic — it's rewiring how organizations work. McKinsey's State of AI survey shows the share of organizations using AI in at least one business function jumped from fifty percent in 2022 to eighty-eight percent this year. The global AI market is valued at roughly 260 billion dollars in 2025, projected to surpass 1.2 trillion by 2030. But this rapid adoption has a cost nobody talks about enough: according to Anaconda's State of Data Science survey, data scientists spend nearly forty percent of their time on data preparation and cleaning — more than they spend building or training models. So while everyone races to adopt AI, the data feeding these systems is too often the weakest link — and that is exactly the gap this thesis addresses.

Slide 5 — Models Fail Because of Data, Not Algorithms**1:30****acum. 3:50**

La caja azul de la derecha es la definición: apóyate en ella al decirla.

In machine learning practice, most of our effort goes into models — but most production failures actually trace back to the data: missing values, duplicates, class imbalance, distribution drift. And among all data problems, one stands out as the most insidious: data leakage — information about the target reaching the training features in a way that will not exist at prediction time. What makes it dangerous is that it is completely silent. Every other data problem makes your model look worse. Leakage makes it look *better* — you get inflated validation metrics that collapse the moment the model faces real data. This has been documented in medicine, in finance, and across large-scale studies. This thesis sets out to automate, quantify, and make actionable exactly that kind of detection, in a single tool.

Slide 6 — A Motivating Example**1:15****acum. 5:05**

Pausa dramática tras decir "one point zero, zero, zero". Señala la flecha.

To quantify this impact, I wanted to build an example scenario. I trained a logistic regression classifier on a dataset containing one engineered leaky feature. Validation accuracy: one point zero zero zero. Perfect. Any of us would be suspicious of a perfect score — but in a large feature table, with a less extreme leak, this goes unnoticed. After removing the leaky features, the honest accuracy is 0.952. That delta of minus 0.048 is fake performance — the gap between a reproducible result and a broken one. A careful human reviewer might catch this case. But at scale, nobody reviews every feature of every dataset. The framework I will show you detects this automatically — and quantifies the damage.

Slide 7 — The Gap in Existing Tools**0:50****acum. 5:55**

Recorre las tres tarjetas de izquierda a derecha. Ve más rápido que antes.

Of course, data quality tooling already exists: ydata-profiling for descriptive reports, Great Expectations for rule-based validation, Deepchecks for train-test checks. They are mature and widely used — but on four constructed leakage scenarios, they detected at most one out of four, and that single detection required manually authoring a rule. None produces an interpretable readiness verdict or code-level fixes. So my framework complements them: it fills the leakage-detection and actionability gap, rather than replacing general-purpose profiling.

Slide 8 — Objectives and Contributions**1:05****acum. 7:00**

Una frase por tarjeta; los KPI de abajo solo señálos.

Closing that gap is exactly what this thesis sets out to do. This translates into four contributions. One: a unified, automated framework — twenty checks across five dimensions, plus impact analysis, via a command line, a Python SDK, and a web application. Two: a unified Leakage Risk Score combining three statistical signals into one flaggable number per feature. Three: semantic leakage analysis with a large language model, for leaks statistics cannot see. And four: a quantitative benchmark against those three tools. Everything is validated on twelve datasets, covered by 343 unit tests, and the code is open source.

Part 2 — System & Methodology (≈7:40)**Slide 9 — Divider 02****0:10****acum. 7:10**

Transición.

Let me now open the system up and show you how it works.

Slide 10 — System Architecture**1:05****acum. 8:15**

Recorre el diagrama de arriba a abajo con el puntero.

The pipeline needs two inputs: a YAML configuration file and a CSV dataset. The data then flows through five analysis dimensions in parallel — quality, leakage, feature analysis, sufficiency, and drift — twenty checks in total. Two more stages complete the picture: impact analysis, which retrains models with and without suspicious features, and an optional LLM-based semantic module. Everything converges into code-level recommendations and a single readiness score, exported as an HTML report and a JSON file.

Slide 11 — Three Interfaces, One Pipeline**0:45****acum. 9:00**

Señala el código: subraya que son cinco líneas.

Now that you have seen how the pipeline works internally, let us look at how you would actually use it. The same pipeline is exposed through three interfaces: a CLI for batch runs and CI/CD, a Python SDK for notebooks — as you can see, a full analysis is about five lines of code — and a Streamlit web application for non-programmers, which I will demo at the end. There is also a plugin API, so teams can register their own domain-specific checks.

Slide 12 — The 20-Check Catalog**0:25****acum. 9:25**

No leas la tabla. Una frase por idea. Ve rápido, es de referencia.

Here we can see the complete check catalog — I won't read it all. Every check works the same simple way: it passes or fails, and tells you exactly which columns are affected. The leakage row is where the novelty lives, so let's zoom into its fifth element: the unified risk score.

Slide 13 — Unified Leakage Risk Score $L(f)$ **1:30****acum. 10:55**

Este es EL slide técnico. Ve despacio. Señala cada término de la ecuación.

Here is the first main contribution. The problem with single statistics is that each of them can be fooled: correlation misses non-linear leaks, mutual information alone is noisy, and both can be diluted by missing values. So for every feature I combine three normalized signals. Rho just measures how correlated a feature is with the target. I-tilde is mutual information, normalized by the maximum across features. And pi is what I call performance inflation: it trains a tiny model on just that one feature and checks how close it gets to perfect accuracy. The weighted combination gives a score between zero and one. At 0.7 the framework raises a warning; at 0.9, an error.

Slide 14 — Are the Weights Arbitrary? — Ablation**1:00****acum. 11:55**

Anticipa aquí la pregunta típica del tribunal. Señala la fila roja.

A fair question is whether those weights are arbitrary, so I ran a sensitivity analysis on real leaky features from the Titanic dataset, where we are predicting passenger survival. The feature *boat* — the lifeboat number — is a true leak, but its raw correlation is diluted by missing values. Under the default weights it scores 0.74 and is correctly flagged. Under a correlation-heavy scheme it drops to 0.662 — below the threshold, a false negative. Every scheme that gives reasonable weight to mutual information or performance inflation catches it. So the defaults are defensible, not decorative — and they remain fully configurable.

Slide 15 — Semantic Leakage Analysis with an LLM**1:00****acum. 12:55**

Ejemplo `discharge_date`: cuéntalo como una mini-historia.

The second contribution addresses leaks that statistics cannot see at all. Imagine an ICU mortality dataset, where we are predicting whether a patient survives, with a feature called *discharge_date*. Statistically it may look innocent — but semantically it is obvious: you only know the discharge date after the outcome. No statistical test reads feature names. So the framework sends feature names, sample values, and a dataset description to a large language model, which evaluates each feature and returns a risk level, from none to high, a leakage type — temporal, proxy, post-hoc or indirect — and a natural-language rationale explaining why. The module is optional, provider-agnostic, disabled by default, and degrades gracefully if no credentials are present.

Slide 16 — Semantic Module: Live Evaluation Results**1:00****acum. 13:55**

Ya NO es un caveat, es un resultado. Dilo con la misma seguridad que antes, pero ahora con orgullo en vez de disculpa. Señala primero la fila medium, luego la tabla de discrepancias.

To check if this actually works, I tested it on thirty real feature names I labelled by hand — including cases like the *discharge_date* example I just showed you — and ran it live against Claude Haiku 4.5 through AWS Bedrock. At the model's normal sensitivity setting, it correctly separates every risky feature from every safe one, with zero mistakes. If I make it stricter, so it only flags the clearest cases, it makes three small, understandable errors — for example, treating one genuinely ambiguous feature as riskier than it really is. Because this is a real, independent model, and not a script that already knew the answers, these mistakes actually tell us something honest about how it reasons.

Slide 17 — From 20 Checks to One Number**0:45****acum. 14:40**

Chequeo de tiempo: deberías ir por ≈ 14 –15 min aquí.

Finally, all results aggregate into a readiness score from zero to one hundred. Each dimension starts at one hundred and loses fifteen points per error and five per warning; the dimensions are then combined with these weights. Leakage carries the largest weight, 0.35, deliberately: it is the only failure mode that silently inflates results instead of visibly degrading them. The score maps to grades A through F, and every weight and penalty is configurable in the YAML file.

Part 3 — Experimental Results ($\approx 3:40$)

Slide 18 — Divider 03**0:10****acum. 14:50**

Transición.

Does it work? Let us look at the evidence.

Slide 19 — Experimental Setup: 12 Datasets**0:30****acum. 15:20**

Izquierda sintéticos, derecha reales. Frase clave: planted vs. surprises. Ve rápido, la slide se explica sola.

I evaluated the framework on twelve datasets: six synthetic, with defects I planted deliberately, and six real-world datasets, all downloaded from OpenML, the open machine-learning data repository built on the UCI Machine Learning Repository — ranging from a few hundred rows up to forty-eight thousand. The synthetic sets verify the framework detects what we planted; the real sets verify it finds surprises in the wild.

Slide 20 — Readiness Scores Discriminate as Designed**1:00****acum. 16:20**

Señala las dos barras naranjas.

And it discriminates exactly as designed. The clean control scores 98.8, grade A. The dirty dataset — full of quality noise but with no leakage — stays in grade A at 91.2. Only the two datasets with confirmed target leakage drop to grade B: the synthetic leaky dataset at 71.6, and, interestingly, Titanic at 80.6. The four additional UCI datasets all score in the nineties — with Wine Quality pulled down to 88.6 by fifteen percent duplicate rows. Quality noise costs points; leakage costs grades.

Slide 21 — Case Study: Titanic**1:15****acum. 17:35**

La anécdota estrella de la defensa. Cuéntala con calma y algo de humor.

Titanic is a dataset of 1,309 passengers, where the target is whether each one survived — and it is also the most famous teaching dataset in the world. So why does it get a B? The framework found two real leaks. First, *name*: with 1,300 unique values it behaves as a row identifier, and its Cramér's V with survival is 0.999 — flagged as an error. Second, and more interesting, *boat*: the lifeboat number. It scores 0.74 on the unified risk score. Think about what that feature means: you were assigned a lifeboat *because* you survived. It literally encodes the target. Thousands of tutorials train models with this column included — and it takes the framework thirty seconds to prove it is textbook post-hoc leakage. Neither leak is caught by any of the three baseline tools.

Slide 22 — Quantifying the Damage: Impact Analysis**0:45****acum. 18:20***Cierra el círculo con el ejemplo del principio.*

The impact analysis module closes the loop on my opening example. It retrains the same cross-validated model with and without the flagged features. On the leaky dataset: baseline 1.000, cleaned 0.952. That measured gap is direct, quantified evidence of leakage — not just a statistical suspicion — and the recommendations then emit the exact pandas code to drop the offending columns.

Part 4 — Benchmark (≈1:50)**Slide 23 — Divider 04****0:10****acum. 18:30***Transición.*

How does this compare against the tools practitioners already use?

Slide 24 — Feature Coverage vs. Three Established Tools**0:50****acum. 19:20***Sé escrupuloso con el framing: checklist ≠ pipeline; menciona fairness.*

I built a 29-item comparison checklist — note this is a cross-tool measurement instrument, more granular than my own 20-check pipeline; the two numbers measure different things. My framework satisfies all 29 items; the closest alternative, Deepchecks, satisfies 11. One fairness note: these baselines were designed for profiling and validation, not leakage — so this chart measures scope, not quality, and the full checklist is published with the benchmark code so anyone can scrutinize it.

Slide 25 — Leakage Detection — the Decisive Result**0:50****acum. 20:10***Esta tabla es el resultado decisivo: dilo explícitamente.*

The decisive result is this one. Four constructed leakage scenarios, every tool with its default configuration. My framework detects all four automatically. ydata-profiling and Deepchecks detect none. Great Expectations detects one — the ID column — and only because I manually authored a rule for it. This is precisely the gap the thesis set out to fill.

Part 5 — Demo & Conclusions (≈5:05)**Slide 26 — Divider 05****0:05****acum. 20:15***Transición rápida; abre ya la pestaña del navegador con Streamlit.*

Let me show you the framework in action.

Slide 27 — Live Demo: Streamlit**3:00****acum. 23:15**

DEMO EN VIVO. Ten la app ya lanzada y titanic.csv en el escritorio. Guion de respaldo si falla: enseña el vídeo grabado o el informe HTML y di: "In the interest of time, let me show you a pre-recorded run."

[Durante la demo, narra:] I upload titanic.csv and select *survived* as the target. . . the pipeline runs the twenty checks plus impact analysis live — this takes a few seconds. . . Here is the readiness summary: 80.6, grade B, with the per-dimension breakdown. In the leakage tab, you can see *name* flagged as an error and *boat* with a risk score of 0.74, each with an explanation and a suggested fix — and here is the exact pandas line to drop the column. Finally, the full HTML report can be downloaded and shared with the team.

Slide 28 — Conclusions

1:00

acum. 24:15

Una frase por tarjeta, con energía de cierre.

To conclude. First: dataset quality assessment and leakage detection can be automated end to end — one config file plus one CSV gives you a scored, explained, and actionable verdict. Second: the unified risk score detects leaks that no single statistic catches, with ablation-backed weights — four out of four benchmark scenarios versus at most one for existing tools. Third: LLMs extend detection to semantic leaks that are invisible to statistics — and a live evaluation against Claude Haiku 4.5 confirms it actually works. And fourth: the framework found real, previously unflagged issues in classic public datasets. On the engineering side: seventeen development phases, fourteen modules, and 343 tests, all passing.

Slide 29 — Future Work

0:40

acum. 24:55

Rápido; el punto 1 conecta con el resultado del slide 16.

Four directions ahead. First, a larger, multi-provider benchmark, now systematically comparing GPT-4o-mini against Claude Haiku 4.5 and other models. Then: broadening the leakage taxonomy to group and preprocessing leakage; CI/CD integration, using the readiness score as a merge gate — the SDK already makes this a few lines of code; and learning flagging thresholds from data instead of fixing them.

Slide 30 — Thank you

0:20

acum. 25:15

Pausa, sonrío, no salgas corriendo del slide.

The code, the datasets, and the full benchmark are available on GitHub. Thank you very much for your attention — I will be happy to take your questions.

Annex — Preguntas probables del tribunal (con respuesta)

¿Por qué esos pesos en $\mathcal{L}(f)$ (0.35/0.35/0.30)?

(EN) They are near-uniform by design — no single signal dominates. The ablation in the thesis shows the flagging decision is stable across reasonable schemes, and that the one clearly bad idea is over-weighting correlation: it reproduces a false negative on Titanic's *boat* (0.662, below the 0.7 threshold). All weights are configurable in the YAML.

¿La evaluación del módulo LLM es real?

(EN) Yes — it's a live call to Claude Haiku 4.5 through AWS Bedrock, not the deterministic mock. At the framework's default threshold, medium, it's a perfect 1.00/1.00/1.00 on the 30-feature benchmark; at the stricter high threshold it's 0.917/0.846/0.880, with three explainable disagreements. I report both thresholds precisely so the perfect number isn't read as cherry-picked. Azure OpenAI credentials were never obtained for this project; AWS credits were.

¿Por qué el umbral 0.95 en `target_leakage` y 0.7/0.9 en LRS?

(EN) They are conservative defaults chosen to keep false positives low on the clean control (which scores 98.8 with no leakage flags), and they are configurable. Learning calibrated, dataset-size-aware thresholds is listed as future work.

¿Escalabilidad? ¿Coste computacional?

(EN) The heaviest components are mutual information and the single-feature models in $\pi(f)$. The largest dataset evaluated is Adult Census, 48,842 rows, which runs the full pipeline in well under a minute on a laptop. Runtime figures are in the benchmark chapter.

¿Por qué GPT-4o-mini / Claude y no un modelo local?

(EN) Cost and quality: both are cheap, fast small models, and the module only sends feature names, a handful of sample values, and a description — not the dataset. The dispatch layer is genuinely provider-agnostic now, not just in theory: it already supports two independent cloud providers, Azure OpenAI and AWS Bedrock, selected by a single config key, and a local endpoint would plug into the same interface.

¿No es injusto el benchmark 29/29 contra herramientas de otro propósito?

(EN) That is exactly why the thesis includes a fairness note: the checklist measures *scope*, not quality, and it is published with the benchmark code. The claim is not that ml-framework is better at profiling — it is that leakage detection is absent elsewhere, which the 4/4 vs. 0–1/4 table shows directly.

¿Qué pasa con falsos positivos (features legítimamente predictivas)?

(EN) A genuinely predictive feature can score high — that is why flags are warnings with explanations, not automatic deletions. The impact analysis helps disambiguate: a legitimate feature removed causes a real performance drop at deployment too, whereas a leak's performance was never real. Final judgement stays with the practitioner; the framework makes the suspicion visible and quantified.

¿Diferencia entre 20, 21, 23 y 29 checks?

(EN) 20 deterministic pipeline checks; +1 optional semantic check (21); a full run reports 22–23 results because impact analysis adds one per configured model; 29 is the separate cross-tool comparison checklist. The thesis uses 20 for the pipeline and 29 only in the benchmark chapter.

Heart Disease: ¿qué pasó con el target?

(EN) While reconciling numbers across documents I found a genuine data bug: a digit-extracting regex collapsed the OpenML labels into a single class. I fixed the download script, re-ran the

entire pipeline on all datasets, and replaced every affected number — the thesis reports only the corrected, reproducible results (96.8/A).

¿Por qué citar cifras de mercado (McKinsey, Gartner, Statista) al principio de una tesis técnica?

(EN) Purely as scene-setting motivation, with named and dated sources cited on the slide — they are not part of the thesis's empirical contribution. The actual contribution is measured independently in the experimental results and the benchmark chapters, on data I collected and controlled myself, not on third-party market estimates.